

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет прикладной информатики
Образовательная программа Мобильные и сетевые технологии
Направление подготовки 09.03.03 Прикладная информатика

О Т Ч Е Т
по курсовой работе
Тема: «Разработка мобильного приложения «Мессенджер»

Обучающийся: Бархатова Наталья Александровна, К3439

Преподаватель: Шатинский Григорий Сергеевич

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Проектирование архитектуры и базовой навигации приложения	4
2 Реализация паттерна MVVM и управление состоянием	8
2.1 Концепция паттерна MVVM в Android	8
2.2 Реализация ViewModel для профиля пользователя	9
2.3 Реализация ViewModel для настроек приложения	11
2.4 Реактивное обновление UI через LiveData	12
3 Сетевое взаимодействие и локальное хранение данных	14
3.1 Архитектура слоя данных (Data Layer)	14
3.2 Локальное хранилище на базе Room Database	14
3.2.1 Проектирование Entity и DAO	14
3.2.2 Настройка базы данных	17
3.3 Сетевое взаимодействие через Retrofit	18
3.3.1 Настройка HTTP-клиента	18
3.3.2 Определение API-интерфейса	19
3.4 Реализация паттерна Repository	20
3.4.1 Логика кэширования данных	20
3.4.2 Обработка сетевых ошибок	22
3.4.2 RecyclerView и пользовательский Adapter	27
4 Улучшение пользовательского интерфейса и интерактивности	29
4.2 Реализация функции «лайков» сообщений	29
4.3 Загрузка и отображение аватаров пользователей	29
4.4 Обработка пользовательских взаимодействий	30
5 Фоновая синхронизация и системные уведомления	32
5.1 Настройка WorkManager для фоновых задач	32
5.1.1 Создание SyncMessagesWorker	32
5.1.2 Конфигурация периодической синхронизации	37
5.2 Реализация системы уведомлений	38

5.2.1 Создание Notification Channel	38
5.2.2 Формирование и отображение уведомлений	39
5.3 Мониторинг состояния сети и реакция на системные события	40
5.3.1 NetworkMonitor с использованием Flow	40
5.3.2 Автоматическая синхронизация при восстановлении сети	43
ЗАКЛЮЧЕНИЕ	44
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	45

ВВЕДЕНИЕ

В современном мире мобильные приложения играют ключевую роль в коммуникации между людьми. Мессенджеры являются одним из наиболее востребованных типов приложений, обеспечивающих обмен сообщениями в режиме реального времени.

Цель курсовой работы – разработать функциональное мобильное приложение-мессенджер для платформы Android.

Для достижения поставленной цели были сформулированы следующие задачи:

- 1) Спроектировать архитектуру приложения на основе паттерна MVVM с использованием компонентов Android Jetpack.
- 2) Реализовать систему навигации между экранами с применением Navigation Component.
- 3) Организовать управление состоянием приложения через ViewModel и LiveData.
- 4) Разработать систему загрузки данных из сетевого API с использованием библиотеки Retrofit.
- 5) Реализовать локальное хранилище на базе Room Database для обеспечения офлайн-доступа к данным.
- 6) Применить паттерн Repository для централизации работы с источниками данных.
- 7) Разработать пользовательский интерфейс с использованием компонентов Material Design.
- 8) Настроить фоновую синхронизацию данных с использованием WorkManager.
- 9) Реализовать систему уведомлений и запрос необходимых разрешений.
- 10) Обеспечить корректную реакцию приложения на системные события.

1 Проектирование архитектуры и базовой навигации приложения

При проектировании архитектуры приложения был выбран паттерн MVVM (Model-View-ViewModel), рекомендованный компанией Google для разработки Android-приложений. Таким образом будет разделение ответственности между компонентами системы и упрощает тестирование и поддержку кода.

Технологический стек приложения включает:

- Язык программирования: Kotlin – современный язык, официально поддерживаемый для разработки под Android, обеспечивающий безопасность типов и null-safety.
- Минимальная версия Android: API 24 (Android 7.0), что охватывает более 95% активных устройств на рынке.

1.2 Реализация системы навигации

Для организации навигации был использован Navigation Component – часть Android Jetpack, предоставляющая декларативный подход к управлению переходами между экранами.

Была создана главная активность MainActivity, наследующаяся от BaseActivity, которая служит контейнером для фрагментов и управляет навигационным графом.

```
class MainActivity : BaseActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
        Log.i("Lifecycle", "MainActivity onCreate")  
        val navController =  
findNavController(R.id.nav_host_fragment)  
        val bottomNav =  
findViewById<BottomNavigationView>(R.id.bottom_navigation)  
        bottomNav.setupWithNavController(navController)  
    }  
}
```

```
        setupPeriodicSync()
    }
}
```

В методе `onCreate` осуществляется инициализация навигационного контроллера и настройка нижней панели навигации. Метод `findNavController(R.id.nav_host_fragment)` получает экземпляр `NavController`, связанный с навигационным хостом. Метод `setupWithNavController` автоматически синхронизирует состояние `BottomNavigationView` с текущим фрагментом в навигационном графе.

Навигационный граф был определён в XML-ресурсе `nav_graph.xml`, где описаны все доступные экраны приложения и связи между ними.

```
<?xml version="1.0" encoding="utf-8"?>
<navigation
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
app:startDestination="@id/profileFragment">
    <fragment
        android:id="@+id/feedFragment"
        android:name="com.example.myapplication.FeedFragment"
        android:label="@string/feed" />
    <fragment
        android:id="@+id/profileFragment"
        android:name="com.example.myapplication.ProfileFragment"
        android:label="@string/profile" />

```

```

<fragment
    android:id="@+id/settingsFragment"

    android:name="com.example.myapplication.SettingsFragmen
t"

    android:label="@string/settings" />
</navigation>

```

Атрибут `app:startDestination` определяет фрагмент, который будет отображён при первом запуске приложения. В данном случае это `profileFragment` – экран профиля пользователя.

Для удобства работы с навигацией был создан объект `Navigation.kt`, содержащий константы идентификаторов фрагментов.

```

object Navigation {
    val FEED_FRAGMENT = R.id.feedFragment
    val PROFILE_FRAGMENT = R.id.profileFragment
    val SETTINGS_FRAGMENT = R.id.settingsFragment
}

```

Нижняя панель навигации была настроена через меню-ресурс `bottom_nav_menu.xml`, содержащий три элемента, соответствующих трём основным экранам приложения.

```

<?xml version="1.0" encoding="utf-8"?>
<menu
    xmlns:android="http://schemas.android.com/apk/res/andro
id">
    <item
        android:id="@+id/feedFragment"
        android:icon="@drawable/feed"
        android:title="@string/feed" />

```

```

<item
    android:id="@+id/profileFragment"
    android:icon="@drawable/profile"
    android:title="@string/profile" />

<item
    android:id="@+id/settingsFragment"
    android:icon="@drawable/settings"
    android:title="@string/settings" />
</menu>

```

Каждый элемент меню связан с соответствующим фрагментом через атрибут `android:id`, значение которого совпадает с идентификатором фрагмента в навигационном графе.

Основной макет активности был реализован с использованием `CoordinatorLayout`, обеспечивающего координацию между различными компонентами UI.

```

<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <fragment
        android:id="@+id/nav_host_fragment"
        android:name="androidx.navigation.fragment.NavHostFragment"
    >

```

```

        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:defaultNavHost="true"
        app:navGraph="@navigation/nav_graph" />

<com.google.android.material.bottomnavigation.BottomNav
igationView

        android:id="@+id/bottom_navigation"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom"
        app:menu="@menu/bottom_nav_menu" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

Компонент NavHostFragment служит контейнером для фрагментов и управляет их жизненным циклом в соответствии с навигационным графом. Атрибут `app:defaultNavHost="true"` указывает, что данный хост должен перехватывать нажатие системной кнопки "Назад".

2 Реализация паттерна MVVM и управление состоянием

2.1 Концепция паттерна MVVM в Android

Паттерн MVVM (Model-View-ViewModel) разделяет приложение на три основных слоя:

- Model – слой данных, включающий Entity, DAO, Repository и источники данных (сеть, база данных).
- View – слой представления, включающий Activity, Fragment и XML-разметку.
- ViewModel – слой бизнес-логики, служащий посредником между Model и View.

Ключевым преимуществом данного паттерна является то, что ViewModel не содержит ссылок на View, что делает его независимым от жизненного цикла UI-компонентов. ViewModel переживает пересоздание Activity/Fragment при изменении конфигурации устройства (например, поворот экрана), сохраняя состояние приложения.

Для обеспечения реактивного обновления UI была использована библиотека LiveData – lifecycle-aware Observable, автоматически управляющий подписками в зависимости от состояния жизненного цикла компонентов.

2.2 Реализация ViewModel для профиля пользователя

Для управления данными профиля пользователя был создан класс ProfileViewModel, наследующийся от базового класса ViewModel.

В данной реализации используется паттерн backing property: приватные изменяемые поля `_name` и `_status` типа `MutableLiveData` инкапсулируют данные, а публичные неизменяемые свойства `name` и `status` типа `LivData` предоставляют доступ для чтения. Такой подход гарантирует, что только ViewModel может изменять состояние, а View – только наблюдать за ним.

Методы `updateName` и `updateStatus` обновляют соответствующие LiveData-объекты, что автоматически приводит к обновлению UI всех подписанных компонентов. Переопределённый метод `onCleared()` вызывается перед уничтожением ViewModel и служит для освобождения ресурсов. В данном случае выполняется логирование для отслеживания жизненного цикла. Фрагмент ProfileFragment связывается с ViewModel и обеспечивает двустороннюю синхронизацию данных.

Используется `ViewModelProvider(requireActivity())` вместо `ViewModelProvider(this)`. Это обеспечивает создание единственного экземпляра ViewModel на уровне активности, что позволяет сохранять данные при переключении между фрагментами. Для обновления EditText из

ViewModel используется `post {}`, гарантирующий выполнение операции в главном потоке UI после завершения текущего layout pass. Проверка `if (nameEdit.text.toString() != name)` предотвращает бесконечный цикл обновлений. Для обновления ViewModel из EditText применяется TextWatcher, отслеживающий изменения в текстовых полях и вызывающий соответствующие методы обновления в `afterTextChanged`.

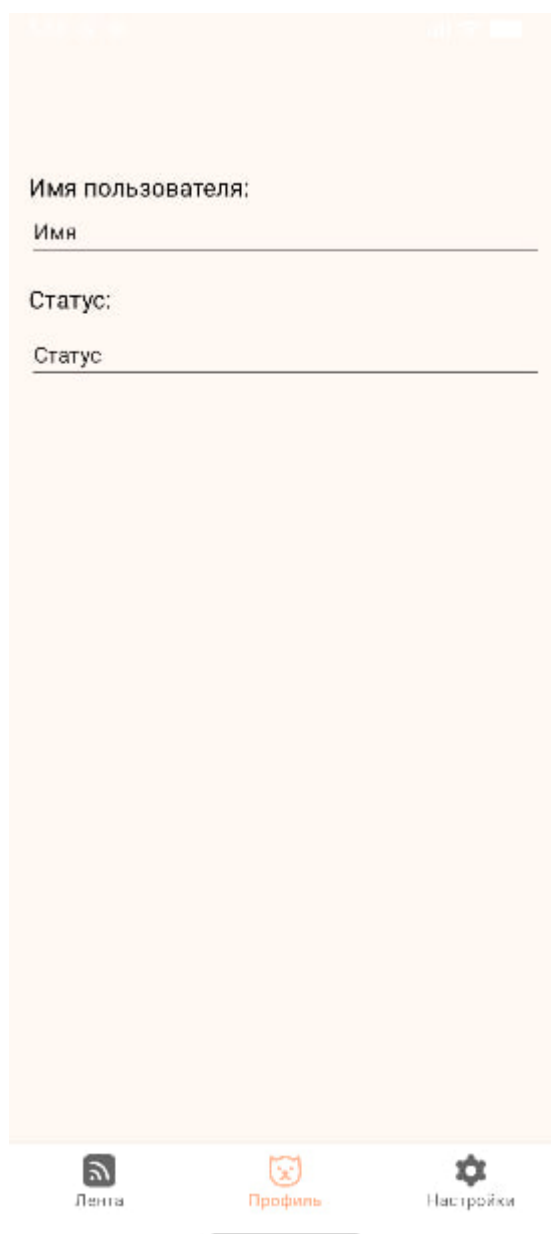


Рисунок 1 – Экран профиля

2.3 Реализация ViewModel для настроек приложения

Аналогичным образом был реализован `SettingsViewModel` для управления настройками приложения. Данная ViewModel управляет

единственным параметром – режимом темы (светлая/тёмная). Метод `setDarkMode` изменяет состояние LiveData-объекта `_darkMode`.

Фрагмент `SettingsFragment` связывается с `ViewModel` и применяет выбранную тему ко всему приложению. При изменении состояния переключателя вызывается метод `setDarkMode`, который обновляет LiveData. Наблюдатель LiveData реагирует на изменение и вызывает `AppCompatActivity.setDefaultNightMode`, применяя выбранную тему ко всему приложению немедленно, без необходимости перезапуска.

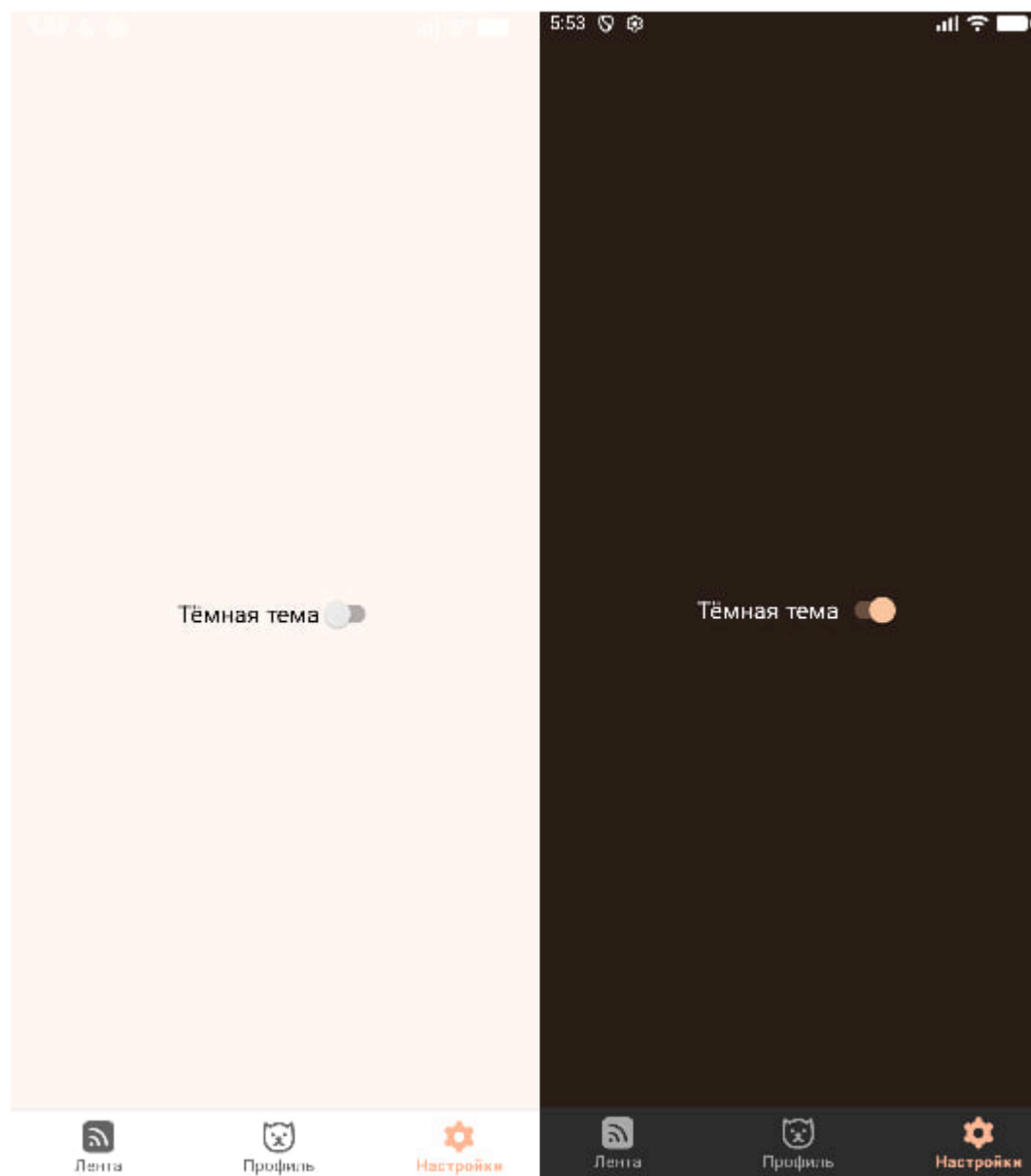


Рисунок 2 – Изменение темы в настройках

2.4 Реактивное обновление UI через LiveData

Особенностью реализованной архитектуры является реактивное обновление пользовательского интерфейса. LiveData автоматически уведомляет наблюдателей об изменениях данных, что исключает необходимость ручного обновления UI.

Механизм работы:

- 1) Пользователь взаимодействует с UI (вводит текст, нажимает переключатель).
- 2) UI компонент вызывает метод ViewModel для обновления данных.
- 3) ViewModel обновляет соответствующий LiveData-объект.
- 4) LiveData автоматически уведомляет всех активных наблюдателей.
- 5) UI обновляется с новыми данными.

LiveData управляет подписками в зависимости от состояния жизненного цикла. Данных профиля и настроек сохраняются при повороте экрана и других изменениях конфигурации устройства.

3 Сетевое взаимодействие и локальное хранение данных

3.1 Архитектура слоя данных (Data Layer)

Слой данных был спроектирован в соответствии с принципом Single Source of Truth – единственного источника истины. Вся работа с данными инкапсулирована в репозитории, которые координируют взаимодействие между локальным хранилищем (Room) и удалённым API (Retrofit).

Архитектура слоя данных включает следующие компоненты:

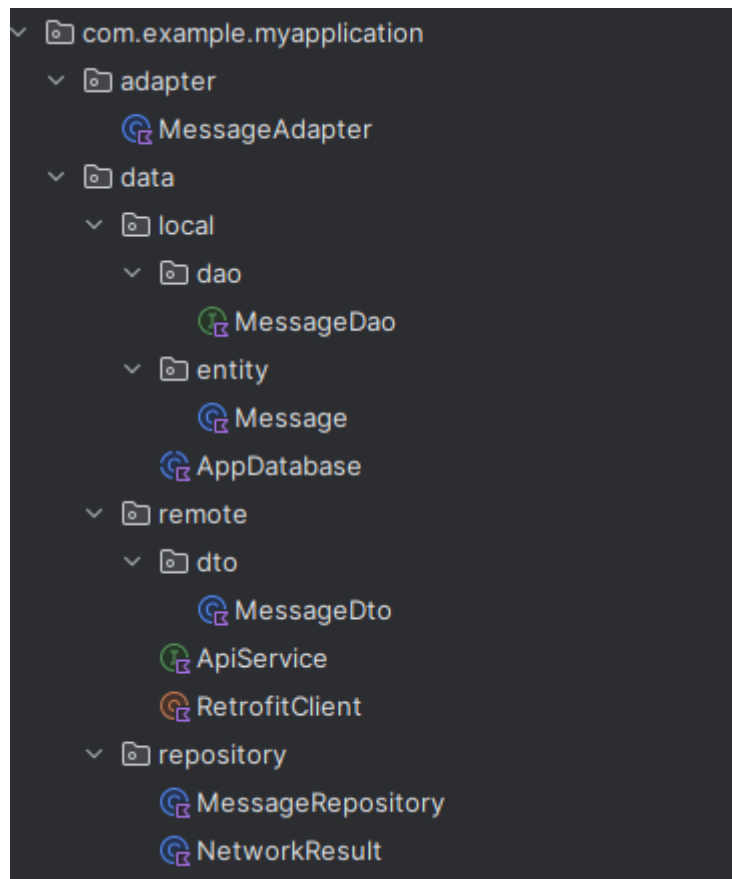


Рисунок 3 – Иерархия классов

ViewModel не зависит от конкретной реализации и взаимодействует только с абстракцией Repository.

3.2 Локальное хранилище на базе Room Database

3.2.1 Проектирование Entity и DAO

Room Database – это библиотека ORM (Object-Relational Mapping), предоставляющая абстракцию над SQLite и упрощающая работу с локальной базой данных.

Была создана Entity-модель Message, представляющая структуру таблицы сообщений.

```
@Entity(tableName = "messages")
data class Message(
    @PrimaryKey
    val id: Int,
    val userId: Int,
    val title: String,
    val body: String,
    val timestamp: Long = System.currentTimeMillis(),
    val isLiked: Boolean = false
)
```

Аннотация `@Entity(tableName = "messages")` указывает Room создать таблицу с именем "messages". Аннотация `@PrimaryKey` определяет поле `id` как первичный ключ таблицы. Поле `timestamp` автоматически заполняется текущим временем при создании объекта, что позволяет отслеживать время кэширования данных. Поле `isLiked` добавлено для реализации функции «лайков» сообщений.

Для работы с данными был создан интерфейс MessageDao (Data Access Object).

```
@Dao
interface MessageDao {

    @Query("SELECT * FROM messages ORDER BY timestamp DESC")
    fun getAllMessages(): Flow<List<Message>>

    @Query("SELECT * FROM messages WHERE id = :messageId")
```

```

suspend fun getMessageById(messageId: Int): Message?

@Insert(onConflict = OnConflictStrategy.REPLACE)
suspend fun insertMessages(messages: List<Message>)

@Insert(onConflict = OnConflictStrategy.REPLACE)
suspend fun insertMessage(message: Message)

@Delete
suspend fun deleteMessage(message: Message)

@Query("DELETE FROM messages")
suspend fun deleteAllMessages()

@Query("SELECT COUNT(*) FROM messages")
suspend fun getMessagesCount(): Int

@Query("UPDATE messages SET isLiked = :isLiked WHERE
id = :messageId")
suspend fun updateLikeStatus(messageId: Int,
isLiked: Boolean)
}

```

Аннотация `@Dao` помечает интерфейс как объект доступа к данным. Room автоматически генерирует реализацию данного интерфейса во время компиляции.

Метод `getAllMessages()` возвращает `Flow<List<Message>>` – реактивный поток данных, автоматически эмитирующий новые значения при любых изменениях в таблице.

3.2.2 Настройка базы данных

Класс AppDatabase является абстрактным классом, наследующимся от RoomDatabase и служащим точкой доступа к базе данных.

```
@Database(
    entities = [Message::class],
    version = 2,
    exportSchema = false
)
abstract class AppDatabase : RoomDatabase() {

    abstract fun messageDao(): MessageDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "messenger_database"
                )
                    .fallbackToDestructiveMigration()
                    .build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

```
}
```

Был реализован паттерн Singleton с использованием Double-Checked Locking для обеспечения единственного экземпляра базы данных во всём приложении. Аннотация `@Volatile` гарантирует видимость изменений переменной `INSTANCE` между потоками. Блок `synchronized(this)` обеспечивает thread-safety при создании экземпляра.

Использование `context.applicationContext` вместо контекста активности предотвращает утечки памяти, так как база данных живёт на протяжении всего жизненного цикла приложения.

Метод `fallbackToDestructiveMigration()` указывает Room пересоздать базу данных при изменении версии схемы. В production-приложениях рекомендуется реализовывать миграции для сохранения пользовательских данных.

3.3 Сетевое взаимодействие через Retrofit

3.3.1 Настройка HTTP-клиента

Для взаимодействия с REST API был использован Retrofit – type-safe HTTP-клиент для Android. Конфигурация клиента была реализована в объекте `RetrofitClient`.

```
object RetrofitClient {  
  
    private val loggingInterceptor =  
HttpLoggingInterceptor().apply {  
        level = HttpLoggingInterceptor.Level.BODY  
    }  
  
    private val okHttpClient =  
OkHttpClient.Builder()  
        .addInterceptor(loggingInterceptor)  
        .connectTimeout(30, TimeUnit.SECONDS)
```

```

        .readTimeout(30, TimeUnit.SECONDS)
        .writeTimeout(30, TimeUnit.SECONDS)
        .build()

    private val retrofit = Retrofit.Builder()
        .baseUrl(ApiService.BASE_URL)
        .client(okHttpClient)

    .addConverterFactory(GsonConverterFactory.create())
        .build()

    val apiService: ApiService =
retrofit.create(ApiService::class.java)
}

```

HttpLoggingInterceptor с уровнем BODY обеспечивает полное логирование HTTP-запросов и ответов, что критически важно при отладке сетевого взаимодействия. В логи выводятся URL, заголовки, тело запроса и ответа. GsonConverterFactory автоматически преобразует JSON-ответы сервера в Kotlin-объекты и наоборот, используя библиотеку Gson для сериализации/десериализации. Метод retrofit.create(ApiService::class.java) использует механизм Dynamic Proxy для генерации реализации интерфейса ApiService во время выполнения программы.

3.3.2 Определение API-интерфейса

Интерфейс ApiService определяет доступные endpoints API.

```

interface ApiService {
    @GET("posts")
    suspend fun getAllMessages(): List<MessageDto>

    companion object {

```

```

const val BASE_URL =
"https://jsonplaceholder.typicode.com/"
    }
}

```

Аннотация `@GET("posts")` указывает Retrofit выполнить HTTP GET-запрос на endpoint `/posts`. Полный URL формируется как `BASE_URL + endpoint = https://jsonplaceholder.typicode.com/posts`.

Для преобразования данных из сетевого формата в формат базы данных был создан класс `MessageDto` (Data Transfer Object). Аннотация `@SerializedName` связывает поля Kotlin-класса с полями JSON-ответа. Метод `toEntity()` выполняет маппинг DTO в `Entity`, добавляя дополнительное поле `timestamp` для отслеживания времени получения данных.

3.4 Реализация паттерна Repository

3.4.1 Логика кэширования данных

Класс `MessageRepository` реализует паттерн `Repository`, служащий единой точкой доступа к данным для `ViewModel`.

`Repository` инкапсулирует логику работы как с локальным хранилищем (`Room`), так и с удалённым API (`Retrofit`). `ViewModel` взаимодействует только с `Repository`, не зная о конкретных источниках данных.

Свойство `messages` экспонирует `Flow<List<Message>>` из DAO, обеспечивая реактивное обновление UI при любых изменениях в базе данных.

```

suspend fun fetchMessages(forceRefresh: Boolean =
false): NetworkResult<List<Message>> {
    return if (isNetworkAvailable()) {
        fetchMessagesFromNetwork()
    } else {
        Log.w("Repository", "Нет интернет-соединения,
загружаем из базы")
    }
}

```

```

        val cachedMessages = getCachedMessages()
        if (cachedMessages.isNotEmpty()) {
            NetworkResult.Success(cachedMessages)
        } else {
            NetworkResult.Error("Нет интернет-соединения  
и нет кэшированных данных")
        }
    }
}

```

Метод `fetchMessages` реализует стратегию "Network First, Cache Fallback":

- 1) Проверяется наличие интернет-соединения через `ConnectivityManager`.
- 2) Если сеть доступна – выполняется загрузка из API.
- 3) Если сеть недоступна – данные загружаются из локального кэша.
- 4) При отсутствии и сети, и кэша возвращается ошибка.

```

private suspend fun fetchMessagesFromNetwork():
NetworkResult<List<Message>> {
    return try {
        Log.i("Repository", "Загружаем сообщения из  
сети...")
        val response = apiService.getAllMessages()
        val messages = response.map { it.toEntity() }
        messageDao.deleteAllMessages()
        messageDao.insertMessages(messages)

        Log.i("Repository", "Загружено ${messages.size}  
сообщений")
        NetworkResult.Success(messages)
    }
}

```

```

    } catch (e: Exception) {
        Log.e("Repository", "Ошибка загрузки из сети:
${e.message}")

        val cachedMessages = getCacheMessages()
        if (cachedMessages.isNotEmpty()) {
            NetworkResult.Success(cachedMessages)
        } else {
            NetworkResult.Error(
                message = "Ошибка загрузки:
${e.message}",
                exception = e
            )
        }
    }
}

```

При успешной загрузке из сети выполняется следующая последовательность операций:

- 1) API-запрос через `apiService.getAllMessages()` (выполняется асинхронно).
- 2) Маппинг списка DTO в список Entity через `map { it.toEntity() }`.
- 3) Очистка таблицы через `deleteAllMessages()`.
- 4) Вставка новых данных через `insertMessages()`.

3.4.2 Обработка сетевых ошибок

При возникновении исключения во время сетевого запроса Repository пытается загрузить данные из кэша. Если кэш содержит данные – возвращается Success с кэшированными сообщениями, иначе – Error с описанием проблемы.

Метод `toggleLike` обновляет статус «лайка» конкретного сообщения в базе данных.

3.5 Интеграция с ViewModel и отображение данных

3.5.1 FeedViewModel для управления лентой

Были созданы четыре LiveData-объекта для управления различными состояниями UI:

- `messages` – список сообщений для отображения
- `isLoading` – индикатор первоначальной загрузки
- `isRefreshing` – индикатор обновления через `SwipeRefresh`
- `error` – сообщение об ошибке

Метод `observeMessages` подписывается на `Flow` из `Repository` и преобразует поток в `LiveData`. Оператор `catch` перехватывает любые ошибки `Flow`, предотвращая краш приложения. Оператор `collect` запускает `Flow` и обновляет `LiveData` при каждом новом значении.

Метод `loadMessages` выполняет загрузку данных с индикацией состояния загрузки. При успешной загрузке данные автоматически попадают в базу и через `Flow` обновляют `LiveData _messages`. При ошибке устанавливается соответствующее сообщение в `_error`.

Метод `refreshMessages` аналогичен `loadMessages`, но использует отдельный флаг `_isRefreshing` для управления индикатором `SwipeRefreshLayout`. Параметр `forceRefresh = true` принуждает `Repository` игнорировать кэш и загружать данные из сети.

Метод `toggleLike` инвертирует текущее состояние лайка и обновляет базу данных. Обновление происходит асинхронно в корутине, не блокируя UI поток.

```
class FeedViewModel(application: Application) :  
    AndroidViewModel(application) {  
  
        private val repository =  
            MessageRepository(application.applicationContext)
```

```

        private val _messages =
MutableLiveData<List<Message>>()

        val messages: LiveData<List<Message>> get() =
        _messages

        private val _isLoading =
MutableLiveData<Boolean>()

        val isLoading: LiveData<Boolean> get() =
        _isLoading

        private val _error = MutableLiveData<String?>()
        val error: LiveData<String?> get() = _error

        private val _isRefreshing =
MutableLiveData<Boolean>()

        val isRefreshing: LiveData<Boolean> get() =
        _isRefreshing

        init {
            Log.i("ViewModel", "FeedViewModel created")
            observeMessages()
            loadMessages()
        }

        private fun observeMessages() {
            viewModelScope.launch {
                repository.messages
                    .catch { e ->
                        Log.e("ViewModel", "Ошибка
Flow: ${e.message}")

```

```

        _error.value = "Ошибка загрузки
данных: ${e.message}"
    }
    .collect { messageList ->
        _messages.value = messageList
        Log.i("ViewModel", "Обновлено
сообщений: ${messageList.size}")
    }
}

}

fun loadMessages(forceRefresh: Boolean = false)
{
    viewModelScope.launch {
        _isLoading.value = true
        _error.value = null

        when (val result =
repository.fetchMessages(forceRefresh)) {
            is NetworkResult.Success -> {
                Log.i("ViewModel", "Успешно
загружено ${result.data.size} сообщений")
            }
            is NetworkResult.Error -> {
                Log.e("ViewModel", "Ошибка:
${result.message}")
                _error.value = result.message
            }
            is NetworkResult.Loading -> {
            }
        }
    }
}

```

```

    }

    _isLoading.value = false
}

}

fun refreshMessages() {
    viewModelScope.launch {
        _isRefreshing.value = true
        _error.value = null

        when (val result =
repository.fetchMessages(forceRefresh = true)) {
            is NetworkResult.Success -> {
                Log.i("ViewModel", "Данные
обновлены")
            }
            is NetworkResult.Error -> {
                _error.value = result.message
            }
            is NetworkResult.Loading -> {}
        }

        _isRefreshing.value = false
    }
}

fun toggleLike(messageId: Int,
currentLikeStatus: Boolean) {
    viewModelScope.launch {

```

```

        repository.toggleLike(messageId,
!currentLikeStatus)

        Log.i("ViewModel", "Лайк изменён:
$messageId -> ${!currentLikeStatus}")
    }
}

override fun onCleared() {
    super.onCleared()
    Log.i("ViewModel", "FeedViewModel cleared")
}

```

3.4.2 RecyclerView и пользовательский Adapter

Для отображения списка сообщений был использован RecyclerView – высокопроизводительный компонент для работы с большими списками данных.

Был создан класс MessageAdapter, наследующийся от ListAdapter – современной альтернативы традиционного RecyclerView.Adapter с встроенной поддержкой DiffUtil.

ListAdapter автоматически вычисляет разницу между старым и новым списком, вызывая только необходимые методы обновления (notifyItemChanged, notifyItemInserted, и т.д.). Это обеспечивает плавную анимацию изменений и экономит ресурсы.

Класс принимает два callback-a:

- onItemClick – для обработки клика на сообщение
- onLikeClick – для обработки клика на кнопку лайка

ViewHolder кэширует ссылки на View, избегая дорогостоящих вызовов findViewById при каждом биндинге данных. Метод bind заполняет View данными из модели Message.

MessageDiffCallback определяет логику сравнения элементов:

- areItemsTheSame сравнивает по ID
- areContentsTheSame сравнивает весь объект

Поскольку Message является data class, метод equals генерируется автоматически и сравнивает все поля, включая isLiked. При изменении лайка DiffUtil обнаружит изменение и обновит только этот элемент списка с анимацией.



Рисунок 4 – Список сообщений с аватарами

4 Улучшение пользовательского интерфейса и интерактивности

4.2 Реализация функции «лайков» сообщений

Функция лайков была реализована с сохранением состояния в локальной базе данных. Иконка лайка меняет свое состояние в зависимости от поля `isLiked` в модели. Используются системные иконки Android: `star_big_on` для лайкнутого сообщения и `star_big_off` для обычного состояния. При клике на иконку вызывается callback `onLikeClick`, переданный в конструктор адаптера. Fragment передает клик в `ViewModel`, которая обновляет базу данных. Изменение в базе триггерит обновление Flow, `DiffUtil` обнаруживает изменение в поле `isLiked` и обновляет только этот элемент списка с анимацией.

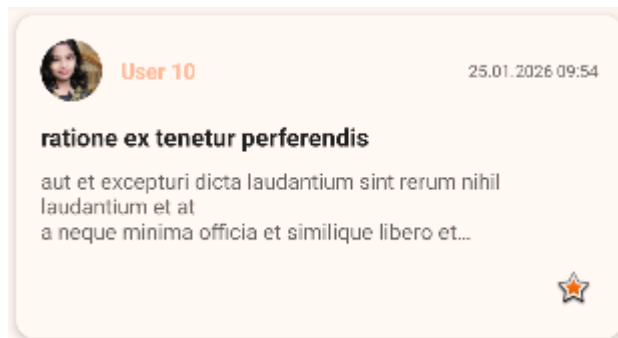


Рисунок 5 – Лайк на сообщение

4.3 Загрузка и отображение аватаров пользователей

Для загрузки аватаров пользователей была использована библиотека Coil – современная библиотека загрузки изображений для Kotlin и Jetpack Compose.

```
val avatarUrl =  
"https://i.pravatar.cc/150?u=${message.userId}"  
  
ivAvatar.load(avatarUrl) {  
    crossfade(true)  
    placeholder(R.drawable.profile)  
    error(R.drawable.profile)  
    transformations(CircleCropTransformation())  
}
```

URL формируется на основе userId: [https://i.pravatar.cc/150?u=\\${userId}](https://i.pravatar.cc/150?u=${userId}).
Сервис pravatar.cc предоставляет случайные аватары с детерминированным выбором по параметру u, что гарантирует, что у одного пользователя всегда будет один и тот же аватар.

Метод load принимает конфигурационную lambda:

- crossfade(true) – плавное появление изображения
- placeholder(R.drawable.profile) – отображение заглушки во время загрузки
- error(R.drawable.profile) – отображение заглушки при ошибке
- transformations(CircleCropTransformation()) – обрезка изображения в круг

4.4 Обработка пользовательских взаимодействий

Были реализованы различные виды пользовательских взаимодействий:

1) Pull-to-refresh через SwipeRefreshLayout

```
private fun setupSwipeRefreshLayout() {  
    swipeRefreshLayout.setOnRefreshListener {  
        viewModel.refreshMessages()  
    }  
  
    swipeRefreshLayout.setColorSchemeResources(  
        R.color.colorPrimary,  
        R.color.colorPrimaryVariant,  
        R.color.colorSecondary  
    )  
}
```

При выполнении жеста "swipe down" вызывается `viewModel.refreshMessages()`. Состояние индикатора синхронизируется с `LiveData isRefreshing`.

2. Клик на элемент списка

При клике на сообщение показывается Toast с заголовком. В реальном приложении здесь можно открыть детальный экран сообщения или диалог.

3. Обработка ошибок с кнопкой повтора

При ошибке загрузки отображается layout с описанием ошибки и кнопкой "Повторить", позволяющей пользователю инициировать новую попытку загрузки.

4. Обновление через меню Toolbar

В Toolbar добавлен пункт меню "Обновить", дублирующий функциональность FAB для доступности.

5 Фоновая синхронизация и системные уведомления

5.1 Настройка WorkManager для фоновых задач

5.1.1 Создание SyncMessagesWorker

Для выполнения фоновой синхронизации данных был использован WorkManager – компонент Android Jetpack, обеспечивающий гарантированное выполнение отложенных задач даже после перезагрузки устройства.

Был создан класс SyncMessagesWorker, наследующийся от CoroutineWorker.

```
package com.example.myapplication.worker

import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.content.Context
import android.content.Intent
import android.os.Build
import android.util.Log
import androidx.core.app.NotificationCompat
import androidx.work.CoroutineWorker
import androidx.work.OneTimeWorkRequestBuilder
import androidx.work.WorkManager
import androidx.work.WorkerParameters
import com.example.myapplication.MainActivity
import com.example.myapplication.R
import
com.example.myapplication.data.repository.MessageRepos
itory
```

```

import
com.example.myapplication.data.repository.NetworkResult

import java.util.concurrent.TimeUnit

class SyncMessagesWorker(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker(context, params) {

    private val repository =
MessageRepository(context)

    companion object {
        const val CHANNEL_ID = "sync_channel"
        const val NOTIFICATION_ID = 1001
        const val WORK_NAME = "sync_messages_work"
    }

    override suspend fun doWork(): Result {
        Log.i("SyncWorker", "ТЕСТ: синхронизация
началась")

        return try {
            when (val result =
repository.fetchMessages(forceRefresh = true)) {
                is NetworkResult.Success -> {
                    val count = result.data.size
                    Log.i("SyncWorker", "ТЕСТ:
получено $count сообщений")

```

```

        showNotification(
            title = "Тестовая
синхронизация",
            message = "Получено
сообщений: $count"
        )

        scheduleNextTestRun()
        Result.success()
    }

    is NetworkResult.Error -> {
        Log.e("SyncWorker", "ТЕСТ:
ошибка ${result.message}")
        scheduleNextTestRun()
        Result.retry()
    }

    is NetworkResult.Loading -> {
        scheduleNextTestRun()
        Result.retry()
    }
}

} catch (e: Exception) {
    Log.e("SyncWorker", "ТЕСТ: исключение
${e.message}")
    scheduleNextTestRun()
    Result.failure()
}

```

```

    }

    private fun scheduleNextTestRun() {
        val request =
OneTimeWorkRequestBuilder<SyncMessagesWorker>()
            .setInitialDelay(1, TimeUnit.MINUTES)
            .build()

WorkManager.getInstance(applicationContext).enqueue(request)

        Log.i("SyncWorker", "ТЕСТ: следующий запуск
через 1 минуту")
    }

    private fun showNotification(title: String,
message: String) {
        val notificationManager =

applicationContext.getSystemService(Context.NOTIFICATIO
N_SERVICE)

        as NotificationManager

        if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                CHANNEL_ID,
                "Тестовая синхронизация",
                NotificationManager.IMPORTANCE_DEFAULT

```

```

        )

notificationManager.createNotificationChannel(channel)
    }

    val intent = Intent(applicationContext,
MainActivity::class.java).apply {
        flags = Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK
    }

    val pendingIntent =
PendingIntent.getActivity(
        applicationContext,
        0,
        intent,
        PendingIntent.FLAG_UPDATE_CURRENT or
PendingIntent.FLAG_IMMUTABLE
    )

    val notification =
NotificationCompat.Builder(applicationContext,
CHANNEL_ID)
        .setSmallIcon(R.drawable.feed)
        .setContentTitle(title)
        .setContentText(message)
        .setAutoCancel(true)
        .setContentIntent(pendingIntent)
        .build()

```

```

        notificationManager.notify(NOTIFICATION_ID,
notification)
    }
}

```

CoroutineWorker предоставляет корутинную версию Worker, позволяя использовать suspend функции внутри метода doWork(). Метод автоматически выполняется в фоновом Dispatcher.

Метод doWork() возвращает Result, указывающий на результат выполнения задачи:

- Result.success() – задача выполнена успешно
- Result.retry() – произошла временная ошибка, нужно повторить
- Result.failure() – критическая ошибка, повтор невозможен

При успешной синхронизации вызывается метод showNotification() для уведомления пользователя.

5.1.2 Конфигурация периодической синхронизации

Для запуска периодической синхронизации был создан Application класс MessengerApplication.

```

private fun setupPeriodicSync() {
    val constraints = Constraints.Builder()

.setRequiredNetworkType(NetworkType.CONNECTED)
    .build()

    val syncRequest =
PeriodicWorkRequestBuilder<SyncMessagesWorker>(
    repeatInterval = 15,
    repeatIntervalTimeUnit = TimeUnit.MINUTES
)
    .setConstraints(constraints)
}

```

```

        .setBackoffCriteria(
            BackoffPolicy.LINEAR,
            WorkRequest.MIN_BACKOFF_MILLIS,
            TimeUnit.MILLISECONDS
        )
        .build()

WorkManager.getInstance(this).enqueueUniquePeriodicWork(
    SyncMessagesWorker.WORK_NAME,
    ExistingPeriodicWorkPolicy.KEEP,
    syncRequest
)

Log.i("WorkManager", "Периодическая
синхронизация настроена")
}

```

Примечание: в финальной версии используется тестовый Worker с интервалом 1 минута для демонстрационных целей (см. `startTestWorker` в `MainActivity`), но для production рекомендуется периодическая синхронизация через `PeriodicWorkRequest`. `PeriodicWorkRequestBuilder` создаёт запрос на периодическое выполнение с минимальным интервалом 15 минут (ограничение Android для периодических задач).

5.2 Реализация системы уведомлений

5.2.1 Создание Notification Channel

Начиная с Android 8.0 (API 26) уведомления должны отправляться через Notification Channel.

NotificationChannel группирует уведомления одного типа, позволяя пользователю контролировать их настройки (звук, вибрация, приоритет) централизованно в системных настройках.

IMPORTANCE_DEFAULT обеспечивает стандартный приоритет: уведомления отображаются, но не прерывают пользователя звуком в режиме "Не беспокоить".

5.2.2 Формирование и отображение уведомлений

PendingIntent создаёт отложенное намерение, которое система может выполнить от имени приложения. При клике на уведомление открывается MainActivity.

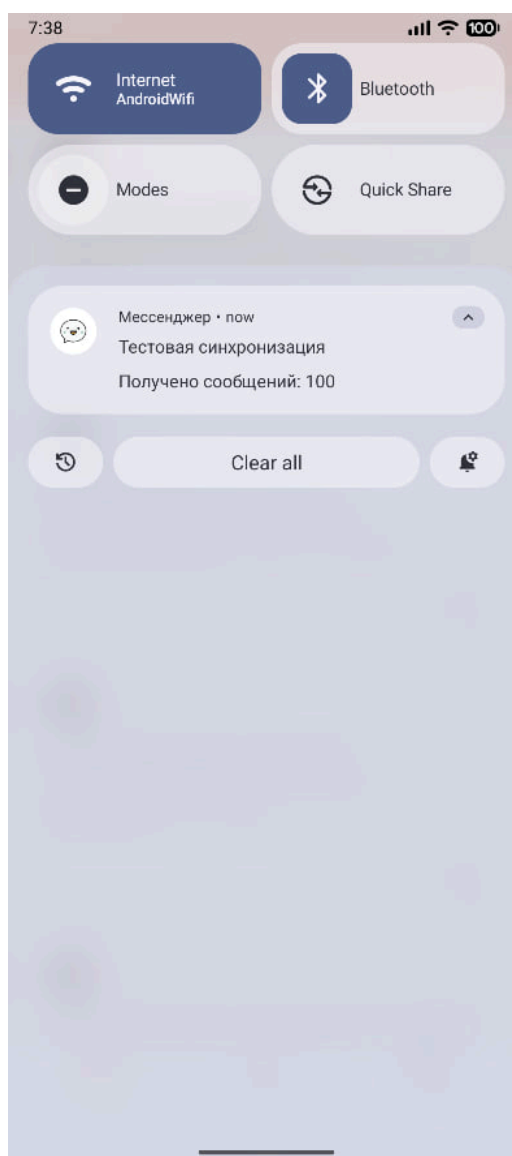


Рисунок 6 – Получение уведомления о синхронизации

5.3 Мониторинг состояния сети и реакция на системные события

5.3.1 NetworkMonitor с использованием Flow

Для отслеживания изменений состояния сети был создан класс NetworkMonitor, использующий ConnectivityManager.NetworkCallback.

```
sealed class NetworkStatus {
    object Available : NetworkStatus()
    object Unavailable : NetworkStatus()
}

class NetworkMonitor(context: Context) {

    private val connectivityManager =
context.getSystemService(Context.CONNECTIVITY_SERVICE)
    as ConnectivityManager

    val networkStatus: Flow<NetworkStatus> =
callbackFlow {
        val networkCallback = object :
ConnectivityManager.NetworkCallback() {

            override fun onAvailable(network:
Network) {
                super.onAvailable(network)
                Log.i("NetworkMonitor", "Сеть
доступна")

                trySend(NetworkStatus.Available)
            }

            override fun onLost(network: Network) {
                super.onLost(network)
            }
        }
    }
}
```

```

        Log.i("NetworkMonitor", "Сеть
потеряна")

        trySend(NetworkStatus.Unavailable)
    }

    override fun onCapabilitiesChanged(
        network: Network,
        networkCapabilities:
NetworkCapabilities
    ) {

        super.onCapabilitiesChanged(network,
networkCapabilities)

        val hasInternet =
networkCapabilities.hasCapability(
NetworkCapabilities.NET_CAPABILITY_INTERNET
        )

        if (hasInternet) {
            Log.i("NetworkMonitor",
"Интернет доступен")

            trySend(NetworkStatus.Available)
        } else {
            Log.i("NetworkMonitor",
"Интернет недоступен")

            trySend(NetworkStatus.Unavailable)
        }
    }

```

```

        }
    }

    val networkRequest =
NetworkRequest.Builder()

.addCapability(NetworkCapabilities.NET_CAPABILITY_INTE
NET)

    .build()

connectivityManager.registerNetworkCallback(networkReq
uest, networkCallback)

    val isConnected = isNetworkAvailable()
        trySend(if (isConnected)
NetworkStatus.Available else
NetworkStatus.Unavailable)

        awaitClose {
            Log.i("NetworkMonitor", "Отменяем
NetworkCallback")

connectivityManager.unregisterNetworkCallback(networkC
allback)

        }

    }.distinctUntilChanged()

```

```

        fun isNetworkAvailable(): Boolean {
            val network =
connectivityManager.activeNetwork ?: return false
            val capabilities =
connectivityManager.getNetworkCapabilities(network) ?:
return false
            return
capabilities.hasCapability(NetworkCapabilities.NET_CAP
ABILITY_INTERNET)
        }
    }
}

```

sealed class NetworkStatus обеспечивает type-safe представление состояния сети с двумя возможными значениями: Available и Unavailable.

callbackFlow создаёт холодный Flow из callback-based API. NetworkCallback регистрируется в ConnectivityManager и отправляет события в Flow через trySend().

Методы callback:

- onAvailable() – сеть стала доступна
- onLost() – сеть потеряна
- onCapabilitiesChanged() – изменились возможности сети (например, переключение с WiFi на мобильные данные)

distinctUntilChanged() фильтрует повторяющиеся значения, предотвращая избыточные обновления UI.

5.3.2 Автоматическая синхронизация при восстановлении сети

В FeedFragment была реализована подписка на изменения состояния сети. При подключении к сети отображается Snackbar с сообщением и автоматически запускается загрузка данных, если список пуст. При отключении сети показывается уведомление о переходе в офлайн-режим. viewLifecycleOwner.lifecycleScope.launch запускает корутину, привязанную к

жизненному циклу View фрагмента. Корутина автоматически отменяется при уничтожении View.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана архитектура и реализован прототип мобильного приложения-мессенджера для платформы Android. В процессе работы выполнен анализ требований, спроектирована архитектура приложения и реализованы основные функциональные компоненты, включая пользовательский интерфейс, слой данных и механизмы фоновой обработки.

В качестве архитектурного подхода был использован паттерн MVVM с применением компонентов Android Jetpack. Реализована навигация между экранами с использованием Navigation Component, управление состоянием интерфейса через ViewModel и LiveData, локальное хранение данных на базе Room и сетевое взаимодействие через Retrofit. Паттерн Repository позволил объединить работу с локальным и удалённым источниками данных и обеспечить корректную загрузку информации при отсутствии сетевого соединения.

В рамках приложения реализованы экран ленты сообщений на основе RecyclerView, функция отметки сообщений, загрузка пользовательских аватаров, обновление данных жестом pull-to-refresh, фоновая синхронизация с использованием WorkManager и система уведомлений с учётом требований актуальных версий Android. Достигнутая цель курсовой работы подтверждает возможность применения выбранных архитектурных и технологических решений для разработки мобильных приложений обмена сообщениями и их последующего функционального расширения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Android Developers. Официальная документация платформы Android : сайт. — URL: <https://developer.android.com/> (дата обращения: 05.10.2025).
- 2) Android Developers. Guide to app architecture : сайт. — URL: <https://developer.android.com/topic/architecture> (дата обращения: 05.10.2025).
- 3) Android Developers. Navigation Component : сайт. — URL: <https://developer.android.com/guide/navigation> (дата обращения: 05.10.2025).
- 4) Square, Inc. Retrofit — Type-safe HTTP client for Android : сайт. — URL: <https://square.github.io/retrofit/> (дата обращения: 30.10.2025).
- 5) Android Developers. Room Persistence Library : сайт. — URL: <https://developer.android.com/training/data-storage/room> (дата обращения: 30.11.2025).
- 6) Android Developers. WorkManager — Background processing : сайт. — URL: <https://developer.android.com/topic/libraries/architecture/workmanager> (дата обращения: 09.11.2025).
- 7) JSONPlaceholder. Free fake API for testing and prototyping : сайт. — URL: <https://jsonplaceholder.typicode.com/> (дата обращения: 21.12.2025).